Module 3 - Port Scanning and Service Enumeration

3.1 - Tool-Stack Overview (Deep Dive)

Before you fire the first SYN packet you need to decide **which engine** does what, how the pieces talk to each other, and how to swap them out if corporate egress filters say “no.”

3.1.1 Layer-by-Layer Architecture

Layer / Task	Primary Tool	Why we chose it (pros)	When to swap (cons)
High-speed discovery scan	**masscan** (C)	• 10 M pps on VPS• SYN only ⇒ kernel stays calm	• Needs raw-socket perms• Can trigger IDS
Stealth discovery fallback	asyncio / trio	• Pure Python → no install• Runs on locked-down VMs	• 5-10 × slower than masscan
Version / service detection	**nmap** w/ -sV -sC	• Thousands of NSE scripts• Parses banners, TLS, SNMP	• Heavy; sequential per host
TLS / SSH deep fingerprint	tls-client or tls-scan, asyncssh	• JA3,SNI,ALPN, key bits	• Extra binaries; *tls-scan* ≈1 MB though
Data store	SQLite + Pandas	• Single portable file• SQL & DataFrame interfaces	• Multi-writer concurrency is poor
Rate/identity control	token_bucket.py (custom)	• Works with every async lib	• Manual tuning needed

> **Golden rule: Scan fast → enumerate slow.**
> Masscan (or async connect) is the *filter*; nmap & friends run **only** against the handful of open ports.

3.1.2 Environment “Sanity Script”

Create `env_check.py` and run it before every engagement:

```python
#!/usr/bin/env python3
"""
env_check.py — Verifies tool versions, raw-socket capability and
Python backlog limits so you don't find out mid-scan that masscan
drops to 300 pps or that the kernel is killing async sockets.
"""
import shutil, os, subprocess, sys, resource

# ---- 1. Binaries ------------------------------------------------------
need = ["masscan", "nmap", "openssl"]        # tls-scan optional
for bin in need:
    if not shutil.which(bin):
        sys.exit(f"[-] Missing binary: {bin}")

# ---- 2. Raw-socket privilege (Linux only) ----------------------------
if os.geteuid() != 0:
    try:
        subprocess.check_call(["masscan", "--ping", "-p0-0", "127.0.0.1"],
                              stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
    except subprocess.CalledProcessError:
        sys.exit("[-] masscan cannot open raw sockets — run as root or set CAP_NET_RAW")

# ---- 3. File-descriptor limit ----------------------------------------
soft, hard = resource.getrlimit(resource.RLIMIT_NOFILE)
print(f"[i] open-file limit: {soft}/{hard}")
if soft < 4096:
    print("[!] Consider `ulimit -n 8192` for high-concurrency scans.")

# ---- 4. Python version & libs ----------------------------------------
```

```python
req = {"trio": "0.24", "httpx": "0.27", "asyncssh": "2.14"}
for mod, ver in req.items():
    try:
        m = __import__(mod)
        from packaging.version import parse
        if parse(m.__version__) < parse(ver):
            print(f"[!] {mod} {m.__version__} < required {ver}")
    except ImportError:
        sys.exit(f"[-] missing pip package {mod}")

print("[+] Environment looks good.")
```

Inline-comment rationale

- Raw-socket check avoids *masscan* silently throttling to TCP-connect mode.
- `RLIMIT_NOFILE` must exceed **concurrency × 2** (socket + epoll file).
- Uses `packaging.version` to warn about stale libraries that lack bug-fixes (e.g., `asyncssh < 2.12` had memory leaks).

3.1.3 Performance Benchmarks (baseline)

Run once on your lab VPS (4 vCPU, 8 GB RAM, 1 Gbps):

Test	Count	Duration	Notes
Masscan SYN → /24, ports 1-1 k	65 535 pkts	**1.1 s**	--rate 60k (≈50 Mb/s)
Async connect() same /24, 1-1 k	65 535 conn	**14 s**	5 k concurrent coroutines
Nmap -sV -sC on 60 open ports	60 sockets	**33 s**	Default NSE set, -T4

> **Take-away:** Masscan gives a 10–12× speed-up in discovery.
> Pure-Python fallback is still < 15 s per /24, acceptable for stealth jobs.

3.1.4 Choosing the Right Rate

- **Enterprise CIDR (/16)** behind Palo Alto → start `--rate 5 k` and increase until IPS logs show no blocks.
- **Home-grown IoT net** (fragile TCP stacks) → skip SYN, use Python connect with 200 ms timeout.
- **Cloud (AWS/GCP)** → enable **low-rate** *and* /32 bursts to avoid black-hole lists.

```
# low-and-slow alias example
alias scan24='masscan -p1-65535 --rate 2000 --wait 0 --source-port 61111'
```

3.1.5 Port-group Optimisation

Scanning **all 65 k** ports is often overkill. A pragmatic tiered approach:

1. **T0 "top 1 k"** (nmap top-ports).
2. **T1 "ent-apps"** (3306, 5432, 6379, 27017, 5000-5099).
3. **T2 full range** if banners from T1 suggest custom software.

Store the tier in your discovery file:

```
{"ip":"10.0.0.5","port":5432,"tier":1}
```

Allows the enumerator to decide how deep to go (`--tier-max 1`).

3.1.6 Quick-Swap Abstraction (Python)

Define a **scan driver** interface so you can switch engines mid-engagement:

```python
class Scanner:
    async def discover(self, targets: list[str]) -> list[dict]: ...
class MasscanScanner(Scanner): ...
class AsyncTCPScanner(Scanner): ...

def get_scanner(mode: str) -> Scanner:
    return MasscanScanner() if mode == "raw" else AsyncTCPScanner()
```

Now every downstream script calls `scanner.discover()` and stays agnostic.

3.1.7 Exercises

1. **Capability probe** – Write a script that checks whether the current user can open a raw socket (`socket.SOCK_RAW`) without elevating, and prints “use-masscan” or “use-async”.
2. **Rate auto-tune** – Implement binary-search to find the fastest `--rate` that yields <0.1 % packet loss (masscan prints “x/ retransmits”).
3. **Custom port-tiers** – Generate T0/T1 lists dynamically from the *popularity* column of your Shodan pull (Module 2.5).
4. **CI smoke-test** – GitHub Actions job that spins up `ghcr.io/massedscan/masscan` container, scans 127.0.0.1, asserts JSON output parses.
5. **Heat output** – Measure CPU & NIC utilisation for masscan at 100 k pps vs 10 k pps and graph with `matplotlib`.

3.2 - Fast Discovery Scan

“Find every listening socket—without lighting up every IDS.”

This stage answers a single question: **Which IP ↔ port tuples are reachable?**
Everything else (version, banner, TLS, exploits) will build on this answer.

3.2.1 Discovery Modes

Mode	Packet Type	Speed	Noise	Priv-req	Recommended When
Raw SYN(masscan)	1 SYN + recv SYN/ACK	1–10 M pps	Medium (firewalls log)	CAP_NET_RAW or root	Large CIDRs, short windows
TCP connect() (async)	Full 3-way handshake	30–100 k pps	Low (looks like legit failures)	none	Stealth, host lists, OT networks
ICMP ping sweep	Echo	100 k pps	Varies	CAP_NET_RAW	Quick “is host up?” pre-filter

We implement **two interchangeable scanners**:

- `masscan_driver.py` – raw SYN, JSON line output.
- `connect_driver.py` – pure Python, async connect, resumable.

The calling code decides which driver to load based on capability and ROE.

3.2.2 masscan_driver.py (heavily commented)

```
#!/usr/bin/env python3
"""
masscan_driver.py
~~~~~~~~~~~~~~~~~
Runs masscan against an INI-style *scope* file and streams the results
to NDJSON for later enumeration.

INI sample:
```

```
  [cidrs]
  nets = 192.168.13.0/24, 10.10.10.5
  [ports]
  tiers = t0            # t0=top1k, t1=top10k, full
"""

from __future__ import annotations
import argparse, configparser, json, pathlib, shlex, subprocess, sys, tempfile
from datetime import datetime

TOP1K = ",".join(map(str, range(1, 1025)))     # quick demo; replace w/ real list

def parse_scope(path: pathlib.Path) -> tuple[list[str], str]:
    cfg = configparser.ConfigParser()
    cfg.read(path)
    nets  = [x.strip() for x in cfg["cidrs"]["nets"].split(",")]
    tier  = cfg["ports"].get("tiers", "t0")
    return nets, tier

def tier_to_ports(tier: str) -> str:
    if tier in ("t0", "top", "quick"):
        return TOP1K
    if tier == "t1":
        return TOP1K + ",3306,5432,6379,27017,5000-5099"
    return "1-65535"

def build_cmd(nets: list[str], ports: str, rate: int, out: pathlib.Path) -> list[str]:
    return shlex.split(
        f"masscan -p{ports} --rate {rate} --wait 0 -oJ {out} " + " ".join(nets)
    )

def run_masscan(cmd: list[str]) -> None:
    print("[*] Launching:", " ".join(cmd))
    proc = subprocess.Popen(cmd)
```

```python
    proc.wait()
    if proc.returncode:
        sys.exit("[-] masscan failed")

def json_to_ndjson(src: pathlib.Path, dst: pathlib.Path) -> None:
    with src.open() as sj, dst.open("w") as dj:
        # The output is a single JSON array; stream-parse line-by-line
        for line in sj:
            if line.lstrip().startswith("{"):
                dj.write(line.rstrip(",\n") + "\n")

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("scope", type=pathlib.Path, help="INI file with nets & ports")
    ap.add_argument("--rate", type=int, default=50000, help="packets per second")
    ap.add_argument("--out",  type=pathlib.Path, default=pathlib.Path("mscan.ndjson"))
    args = ap.parse_args()

    nets, tier = parse_scope(args.scope)
    ports = tier_to_ports(tier)

    with tempfile.NamedTemporaryFile(delete=False) as tmp:
        cmd = build_cmd(nets, ports, args.rate, pathlib.Path(tmp.name))
        run_masscan(cmd)
        json_to_ndjson(pathlib.Path(tmp.name), args.out)

    print(f"[+] Discovery scan finished — {args.out}  {args.out.stat().st_size/1e6:.1f} MB")

if __name__ == "__main__":
    main()
```

Inline-comment notes

- Uses `tempfile` so partially written JSON never corrupts destination.
- Tier-to-ports lets ops downgrade to quick scans by editing INI only.
- Masscan JSON is **array with trailing commas**; we stream into valid NDJSON.

3.2.3 connect_driver.py (pure-Python stealth)

```python
#!/usr/bin/env python3
"""
connect_driver.py
~~~~~~~~~~~~~~~~~
TCP connect() scanner—safe on networks where raw SYN triggers alarms.

Features
  • Resumable: keeps checkpoint every N IPs.
  • Concurrency tuned via --workers X.
  • Writes same NDJSON schema as masscan_driver.
"""

from __future__ import annotations
import argparse, asyncio, json, pathlib, random, time
from typing import Iterable
from rich.progress import Progress, BarColumn, TimeRemainingColumn

DEFAULT_PORTS = [80, 443, 22, 3389, 3306, 5432]  # override via CLI

async def probe(ip: str, port: int, timeout: float) -> bool:
    try:
        conn = asyncio.open_connection(ip, port)
        reader, writer = await asyncio.wait_for(conn, timeout)
        writer.close()
```

```python
        await writer.wait_closed()
        return True
    except Exception:
        return False

async def scan_host(ip: str, ports: Iterable[int], sem: asyncio.Semaphore,
                    out_fh, timeout: float):
    for port in ports:
        async with sem:
            if await probe(ip, port, timeout):
                out_fh.write(json.dumps({"ip": ip, "port": port}) + "\n")
        # micro jitter
        await asyncio.sleep(random.uniform(0.01, 0.05))

async def main(cidr_file: pathlib.Path, out: pathlib.Path,
               workers: int, timeout: float, ports: list[int]):

    ips = [line.strip() for line in cidr_file.read_text().splitlines() if line.strip()]
    sem = asyncio.Semaphore(workers)
    with out.open("w") as fh, Progress(
        "{task.description}", BarColumn(), TimeRemainingColumn()
    ) as bar:
        task = bar.add_task("[cyan]connect", total=len(ips))
        for ip in ips:
            await scan_host(ip, ports, sem, fh, timeout)
            bar.advance(task)

if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("targets", type=pathlib.Path, help="list of IPs (one per line)")
    ap.add_argument("--out", default="connect.ndjson", type=pathlib.Path)
    ap.add_argument("-p", "--ports", help="comma list or 'top1k'", default="")
    ap.add_argument("-w", "--workers", type=int, default=800)
    ap.add_argument("--timeout", type=float, default=1.5)
```

```
    args = ap.parse_args()

    port_list = (list(range(1, 1025)) if args.ports == "top1k"
                 else [int(x) for x in args.ports.split(",") if x]) or DEFAULT_PORTS
    asyncio.run(main(args.targets, args.out, args.workers, args.timeout, port_list))
```

Why semaphore?
`asyncio` opens a socket per coroutine; without a maximum we'd exceed OS file-descriptor limits and trigger SYN floods.

3.2.4 Output Schema (uniform)

Every discovery driver writes **one JSON per line**:

```
{"ip": "203.0.113.17", "port": 443}
```

Later enumeration scripts simply:

```
for rec in map(json.loads, Path(source).read_text().splitlines()):
    enumerate(rec["ip"], rec["port"])
```

No conditionals for driver-type.

3.2.5 Merging & Deduplicating Results

You can run both drivers (raw SYN on weekend, TCP connect on weekdays) and merge:

```python
import pandas as pd
df = pd.concat([
    pd.read_json("mscan.ndjson", lines=True),
    pd.read_json("connect.ndjson", lines=True)
]).drop_duplicates()
df.to_json("all_open.ndjson", orient="records", lines=True)
```

3.2.6 Quality-Assurance Metrics

Metric	Target value	Command / script
Packet-loss (masscan)	< 0.5 %	Parse “**retransmits**” line from stdout
SYN-ACK to RST ratio	> 0.9	Capture pcap and use tcp.stream eq 0 filters
Connect() fail-ratio	< 5 % timeouts	Count probe attempts vs successes
FD usage	< 80 % soft limit	`ls /proc/$(pgrep -f connect_driver)/fd

---

3.2.7 Exercises

1. **Fragmentation mode** – Add `--frag 8` to masscan command to bypass some IDS; measure whether open count changes.
2. **CIDR scheduler** – Given 5 /24 networks, parallelise masscan with GNU `parallel` so each runs at 20 k pps simultaneously.
3. **Host liveness pre-filter** – Implement ICMP ping sweep; run TCP connect only against responsive hosts.
4. **Resume flag** – Save last processed IP in a `.state` file; on crash, restart scanning from there.
5. **FP reduction** – For every masscan SYN hit, validate with a follow-up TCP connect before marking “open”.

3.2.8 Completion check

- At least one driver succeeds against your lab /24 in < 30 s.
- NDJSON `all_open.ndjson` contains ip,port pairs only—no dupes.
- Loss/timeout metrics within tolerance.
- Scripts bail gracefully if `stop.txt` present or `Ctrl-C` pressed.

3.3 - Service-Enumeration Escalation

*"You already know **something** is listening—now find out **exactly what**."*

> **Input:** `all_open.ndjson` from 3.2 (one `{"ip": …, "port": …}` per line).
> **Output:** four richly-annotated tables (`nmap` , `tls` , `ssh` , `http`) merged into the master SQLite database **`recon.db`** .

3.3.0 Strategy in 90 seconds

```
1. Read open-port list
2. Bucket targets by protocol hints (80 → HTTP, 22 → SSH, etc.)
3. Launch *light* detectors first (Nmap -sV/-sC) with adaptive timing
4. Route results into protocol-specific deep probers
5. Flatten everything into SQLite
```

Why multi-stage?

- **Time-saving** – You don't waste a full TLS scanner on a telnet port.
- **Fail-soft** – If deep probes crash, the baseline Nmap result still exists.
- **Forensics** – Every escalating step appends columns, never overwrites.

3.3.1 Baseline Enumeration with Nmap

3.3.1-A Wrapper Script (`nmap_baseline.py`)

```python
#!/usr/bin/env python3
"""
nmap_baseline.py
----------------
• Reads all_open.ndjson
• Groups by IP so we launch one Nmap per host
  (faster than per-port invocations)
• Runs -sV (version) + -sC (default NSE scripts)
• Adaptive timing: starts at -T4, falls back to -T3
  if packet loss > 5 %.
• Outputs one XML per host  (nmap_xml/<IP>.xml)
"""

from __future__ import annotations
import json, pathlib, subprocess, shlex, statistics, xml.etree.ElementTree as ET
from rich.progress import track

INFILE   = pathlib.Path("all_open.ndjson")
XML_DIR  = pathlib.Path("nmap_xml"); XML_DIR.mkdir(exist_ok=True)

# --- build {ip: [ports]} dict ----------------------------------------------
targets: dict[str, list[int]] = {}
for line in INFILE.read_text().splitlines():
    rec = json.loads(line)
    targets.setdefault(rec["ip"], []).append(rec["port"])

# --- helper ----------------------------------------------------------------
def run_nmap(ip: str, ports: list[int], speed: int) -> pathlib.Path:
    xml_out = XML_DIR / f"{ip}.xml"
```

```
    port_list = ",".join(map(str, sorted(ports)))
    cmd = f"nmap -sV -sC -T{speed} -p {port_list} -oX {xml_out} {ip}"
    subprocess.run(shlex.split(cmd), stdout=subprocess.DEVNULL)
    return xml_out

# --- adaptive timing logic ---------------------------------------------------
for ip, ports in track(targets.items(), description="Nmap"):
    xml = run_nmap(ip, ports, speed=4)
    # quick loss heuristic: if <25 % of given ports are reported, retry slower
    found = len(ET.parse(xml).findall(".//port[state/@state='open']"))
    if found < 0.75 * len(ports):
        xml.unlink()
        xml = run_nmap(ip, ports, speed=3)

print(f"[+] Baseline XMLs in {XML_DIR}")
```

Comment highlights

- **Single invocation per IP** saves TCP connect overhead versus `-iL` big list in most cases.
- Simple “coverage” check detects IPS dropping packets, auto-retries at `-T3`.

3.3.2 Deep-Dive Probes by Protocol

We chain **three lightweight, async scripts** that read the baseline XML:

```
nmap_xml/ → tls_probe.py   (ports 443, 8443, 993 …)
         → ssh_probe.py    (port 22)
         → http_probe.py   (ports 80, 8080, 8000, 443) for headers & titles
```

3.3.2-A TLS Probe (JA3S, ALPN, Cert CN)

```python
#!/usr/bin/env python3
"""
tls_probe.py — uses tls-client (pure Python TLS stack that mimics Chrome)
to pull server fingerprint without forking openssl.
"""
import json, pathlib, asyncio, csv
from tls_client import Session, TLSClient  # pip install tls-client

PORT_HINT = {443, 8443, 993, 465}
XML_DIR   = pathlib.Path("nmap_xml")
CSV_OUT   = pathlib.Path("tls.csv")

sess = Session(client_identifier="chrome_120")

def parse_xml() -> list[tuple[str, int]]:
    from xml.etree import ElementTree as ET
    for xml in XML_DIR.glob("*.xml"):
        root = ET.parse(xml).getroot()
        ip = root.findtext("host/address[@addrtype='ipv4']")
        for port in root.findall(".//port[state/@state='open']"):
            p = int(port.get("portid"))
            if p in PORT_HINT:
                yield ip, p

async def grab(ip: str, port: int) -> dict | None:
    try:
        r = await sess.get(f"https://{ip}:{port}", timeout=5.0, insecure=True)
        info = r.tls_socket  # tls-client exposes TLS meta here
        return {
            "ip": ip,
            "port": port,
            "ja3s": info.ja3s,
            "alpn": ";".join(info.negotiated_alpn_protocols or []),
```

```
            "cn":   info.peer_certificate["subject"]["CN"] if info.peer_certificate else "",
        }
    except Exception:
        return None

async def main():
    rows = []
    tasks = [grab(ip, p) for ip, p in parse_xml()]
    for coro in asyncio.as_completed(tasks):
        res = await coro
        if res: rows.append(res)
    with CSV_OUT.open("w", newline="") as fh:
        csv.DictWriter(fh, rows[0].keys()).writerows(rows)

asyncio.run(main())
print(f"[+] TLS metadata → {CSV_OUT}")
```

*Why **tls-client**?*
It produces **both** JA3S and ALPN in one round-trip and fakes Chrome JA3 → WAFs treat us as legit browsers.

3.3.2-B SSH Probe (Version, Host-Key bits)

```
#!/usr/bin/env python3
import asyncssh, asyncio, csv, pathlib, xml.etree.ElementTree as ET

CSV = pathlib.Path("ssh.csv")
hosts = []
for xml in pathlib.Path("nmap_xml").glob("*.xml"):
    ip = ET.parse(xml).findtext("host/address[@addrtype='ipv4']")
    if any(p.get("portid") == "22" and p.find("state").get("state") == "open"
           for p in ET.parse(xml).findall(".//port")):
```

```
        hosts.append(ip)

async def probe(ip):
    try:
        async with asyncssh.connect(ip, username="none", known_hosts=None,
                                    login_timeout=2) as conn:
            key = conn.get_server_host_key()
            return [ip, conn.get_server_version(), key.get_algorithm(), key.get_bit_size()]
    except Exception:
        return None

async def main():
    rows = await asyncio.gather(*(probe(ip) for ip in hosts))
    rows = [r for r in rows if r]
    with CSV.open("w", newline="") as fh:
        csv.writer(fh).writerows([["ip", "banner", "key_algo", "bits"], *rows])

asyncio.run(main())
print(f"[+] SSH info → {CSV}")
```

3.3.2-C HTTP Probe (Headers +)

```
#!/usr/bin/env python3
import httpx, asyncio, csv, pathlib, xml.etree.ElementTree as ET, re

CSV = pathlib.Path("http.csv")
PORT_HINT = {80, 8080, 8000, 443}

targets = []
for xml in pathlib.Path("nmap_xml").glob("*.xml"):
    ip = ET.parse(xml).findtext("host/address[@addrtype='ipv4']")
    for prt in ET.parse(xml).findall(".//port[state/@state='open']"):
        if int(prt.get("portid")) in PORT_HINT:
```

```
            targets.append((ip, int(prt.get("portid"))))

async def get(ip, port):
    scheme = "https" if port == 443 else "http"
    url = f"{scheme}://{ip}:{port}/"
    try:
        async with httpx.AsyncClient(http2=True, verify=False, timeout=4) as cli:
            r = await cli.head(url)
            banner = r.headers.get("server", "")
            title = ""
            if not banner:
                r2 = await cli.get(url)
                m = re.search(r"<title.*?>(.*?)</title>", r2.text, re.I | re.S)
                title = m.group(1).strip() if m else ""
            return [ip, port, banner or title]
    except Exception:
        return None

async def main():
    rows = await asyncio.gather(*(get(ip, p) for ip, p in targets))
    rows = [r for r in rows if r]
    with CSV.open("w", newline="") as fh:
        csv.writer(fh).writerows([["ip", "port", "banner"], *rows])

asyncio.run(main())
print(f"[+] HTTP titles/headers → {CSV}")
```

3.3.3 Fusion into SQLite (`recon.db`)

```
#!/usr/bin/env python3
import pandas as pd, sqlite3, pathlib, glob
```

```python
db = sqlite3.connect("recon.db")
# baseline open ports
pd.read_json("all_open.ndjson", lines=True).to_sql("ports", db, if_exists="replace")

# nmap
frames = [pd.read_xml(x, xpath=".//port") for x in glob.glob("nmap_xml/*.xml")]
pd.concat(frames).to_sql("nmap", db, if_exists="replace")

# deep probes
pd.read_csv("tls.csv").to_sql("tls", db, if_exists="replace")
pd.read_csv("ssh.csv").to_sql("ssh", db, if_exists="replace")
pd.read_csv("http.csv").to_sql("http", db, if_exists="replace")

db.close()
print("[+] Service-enumeration tables added to recon.db")
```

Now one query gives everything:

```sql
SELECT p.ip, p.port, n.service, t.cn, h.banner
FROM ports p
LEFT JOIN nmap n ON p.ip=n.host AND p.port=n.portid
LEFT JOIN tls  t ON p.ip=t.ip  AND p.port=t.port
LEFT JOIN http h ON p.ip=h.ip AND p.port=h.port
WHERE service LIKE 'http%'
  AND banner LIKE '%Apache%'
  AND bits < 2048;
```

3.3.4 Safety Nets

- **Per-IP queue**: if a host shows >10 open ports, pause 2 s before nmap to avoid IDS “TCP Port Sweep”.
- **Kill-switch**: every probe script checks for `stop.txt` in cwd; exits immediately if present.
- **Vendor bug lists**: before running NSE scripts like `ssh-auth-methods`, confirm they’re in `safe` category.

3.3.5 Exercises

1. **SMB probe** – Write `smb_probe.py` that uses `impacket.smbclient` to pull `ServerOS` and `Dialect` for ports 445/139.
2. **MySQL handshake** – Craft raw TCP probe: read initial `Server Greeting`, grab version string without authenticating.
3. **“Weird cipher” alert** – SQL query that lists hosts whose TLS favorite cipher is `TLS_RSA_WITH_3DES_EDE_CBC_SHA`.
4. **Elastic resume** – Modify `nmap_baseline.py` to skip IPs if corresponding XML already exists and has `<finished>`.
5. **Graph export** – Use NetworkX to build `ip → port → service` triples, color nodes by top-level protocol.

3.3.6 DONE Checklist

- Nmap XML exists for every host with open ports.
- `tls.csv`, `ssh.csv`, `http.csv` populated correctly.
- All tables merged into `recon.db` with consistent primary keys.
- Scripts respect kill-switch and slow down on high-port counts.

3.4 - TLS, SSH & App-specific

*Adds context to every open port discovered in **3.2** and catalogued by the baseline Nmap pass in **3.3**.*

> **Input:** `nmap_xml/` directory (one XML per host).
> **Output:** three CSV files (`tls.csv`, `ssh.csv`, `http.csv`) that are later merged into **`recon.db`**.
> **Focus:** Fully-commented Python code with explanations **outside** the snippets so you can copy-paste and still understand the reasoning.

3.4.0 Why “deep” probes?

- Banners alone lie (e.g. Apache 2.4 with mod_security shows “Server: nginx”).
- TLS metadata (JA3S, ALPN, certificate CN) identifies WAFs, CDNs, mis-issued certs.
- SSH host-key length reveals obsolete keys (e.g. 1024-bit DSA).
- HTTP “HEAD ➜ GET” dance grabs titles when the `Server` header is missing.

3.4.1 TLS Probe (`tls_probe.py`) — line-by-line walkthrough

Below we use **`tls-client`** (a wrapper around *curl*’s h2 TLS stack) because it can:

- Fake a modern Chrome *client* JA3 (helps bypass naive WAFs).
- Return the ***server*** JA3S, ALPN list, and full parsed cert in **one** call.

```
#!/usr/bin/env python3
"""
tls_probe.py
------------
Pulls deep TLS metadata for every open HTTPS-like port found in the baseline
Nmap XMLs. Results are written to tls.csv

Fields:
  ip, port, ja3s, alpn, cn, cert_not_after
"""

from __future__ import annotations
import asyncio, csv, json, pathlib, re, ssl
from datetime import datetime
from tls_client import Session                # $ pip install tls-client
```

```python
from xml.etree import ElementTree as ET
from rich.progress import Progress, BarColumn, TimeElapsedColumn

################################################################################
# 1.  Build the target list from XML
################################################################################

XML_DIR   = pathlib.Path("nmap_xml")
PORT_HINT = {443, 8443, 993, 465}
targets   = []                              # (ip, port)

for xml in XML_DIR.glob("*.xml"):
    root = ET.parse(xml).getroot()
    ip = root.findtext("host/address[@addrtype='ipv4']")
    for prt in root.findall(".//port[state/@state='open']"):
        port = int(prt.get("portid"))
        if port in PORT_HINT:
            targets.append((ip, port))

################################################################################
# 2.  Single async probe helper
################################################################################

UA_SESS = Session(client_identifier="chrome_120")  # Chrome-like JA3

async def tls_meta(ip: str, port: int) -> dict | None:
    """
    Return dict with TLS attributes or None on any failure.
    • Insecure=True skips certificate validation (targets may use self-signed)
    • One round-trip; tls-client returns .tls_socket metadata.
    """
    # Build URI; use IP as host and omit SNI (server_hostname=None) to avoid
    # triggering virtual-host mismatches when scanning plain IPs.
    url = f"https://{ip}:{port}/"
```

```python
    try:
        r = await UA_SESS.get(url, timeout=6.0, insecure=True)
        info = r.tls_socket                                # custom object

        # tls-client already parsed these for us
        cn      = (info.peer_certificate["subject"].get("CN")
                   if info.peer_certificate else "")
        ja3s    = info.ja3s_hash
        alpn    = ";".join(info.negotiated_alpn_protocols or [])
        expires = info.peer_certificate.get("validTo") if info.peer_certificate else ""
        return {
            "ip": ip, "port": port, "ja3s": ja3s,
            "alpn": alpn, "cn": cn, "cert_not_after": expires,
        }
    except Exception:
        return None            # fail-soft: never raise

##############################################################################
# 3.  Fan-out / gather with concurrency
##############################################################################

async def main(concurrency: int = 250):
    rows = []
    sem  = asyncio.Semaphore(concurrency)

    async def worker(ip, port):
        async with sem:                            # cap open sockets
            res = await tls_meta(ip, port)
            if res:
                rows.append(res)

    # Progress bar for operator feedback
    with Progress("{task.description}", BarColumn(), TimeElapsedColumn()) as bar:
```

```
        task = bar.add_task("[cyan]TLS", total=len(targets))
        await asyncio.gather(*(worker(ip, port) for ip, port in targets))
        bar.update(task, completed=len(targets))

    # Persist CSV
    out = pathlib.Path("tls.csv")
    with out.open("w", newline="") as fh:
        csv.DictWriter(fh, fieldnames=rows[0].keys()).writerows(rows)
    print(f"[+] TLS metadata saved → {out}")

if __name__ == "__main__":
    asyncio.run(main())
```

What every major comment explains

- **Parsing XML once** keeps the probe script decoupled from previous scan logic.
- **Semaphore** ensures we don't exceed 250 live TLS handshakes → no port-flood alarms.
- **fail-soft** principle: *any* exception returns `None` so one broken host never kills the pool.

3.4.2 SSH Probe (`ssh_probe.py`) — what & why explained

```
#!/usr/bin/env python3
"""
ssh_probe.py
------------
Connects to port 22 (or 2222) of every host, captures:
  • banner (OpenSSH_8.0p1 Debian-6)
  • host-key algorithm (rsa, ecdsa-sha2-nistp256 …)
  • key bit-length (1024/2048/4096)
```

```python
Outputs ssh.csv for later JOINs.
"""

import asyncio, asyncssh, csv, pathlib, xml.etree.ElementTree as ET
from rich.progress import Progress

# ------------------------------------------------------------------------ #
# 1. Collect targets (22 + 2222)                                           #
# ------------------------------------------------------------------------ #
XML_DIR = pathlib.Path("nmap_xml")
PORTS   = {22, 2222}
targets = []

for xml in XML_DIR.glob("*.xml"):
    ip = ET.parse(xml).findtext("host/address[@addrtype='ipv4']")
    for p in ET.parse(xml).findall(".//port[state/@state='open']"):
        if int(p.get("portid")) in PORTS:
            targets.append((ip, int(p.get("portid"))))

# ------------------------------------------------------------------------ #
# 2. Non-blocking SSH handshake                                            #
# ------------------------------------------------------------------------ #
async def probe(ip: str, port: int):
    """
    We only need the *handshake*, so:
      • username 'none' (server immediately kicks us)
      • known_hosts=None to skip fingerprint verification
      • login_timeout keeps it snappy
    """
    try:
        async with asyncssh.connect(ip, port=port, username="none",
                                    known_hosts=None, login_timeout=3) as conn:
            key = conn.get_server_host_key()
            return {
```

```
                "ip": ip,
                "port": port,
                "banner": conn.get_server_version(),
                "algo": key.get_algorithm(),
                "bits": key.get_bit_size(),
            }
    except (asyncio.TimeoutError, asyncssh.Error):
        return None

async def main():
    rows, sem = [], asyncio.Semaphore(300)

    async def worker(t):
        async with sem:
            res = await probe(*t)
            if res:
                rows.append(res)

    with Progress("[cyan]SSH", total=len(targets)) as pb:
        await asyncio.gather(*(worker(t) for t in targets))
        pb.advance(len(targets))

    # CSV out
    out = pathlib.Path("ssh.csv")
    with out.open("w", newline="") as fh:
        csv.DictWriter(fh, fieldnames=rows[0].keys()).writerows(rows)
    print(f"[+] SSH details → {out}")

if __name__ == "__main__":
    asyncio.run(main())
```

Key explanation lines

- `known_hosts=None` avoids writing to `~/.ssh/known_hosts`, keeping the VM clean.
- We respect an **overall semaphore** so that slow hosts don't blow up FD count.

3.4.3 HTTP Probe (`http_probe.py`) — headers + `<title>`

Why another HTTP probe after Nmap?

- Nmap only runs a few scripts; we still miss titles when the `Server` header is blank.
- It helps analysts eyeball interesting admin panels quickly.

```
#!/usr/bin/env python3
"""
http_probe.py
-------------
Targets ports 80/443/8080/8000 discovered earlier.
• HEAD first (fast), falls back to GET (title scrape) if headless.
• Skips if body contains 'captcha' to avoid unneeded HTML bloat.
"""

import asyncio, csv, httpx, json, pathlib, re
from rich.progress import track

NDJSON     = pathlib.Path("all_open.ndjson")
OUT_CSV    = pathlib.Path("http.csv")
PORT_HINTS = {80, 443, 8080, 8000, 8443}

targets = [(r["ip"], r["port"])
           for r in map(json.loads, NDJSON.read_text().splitlines())
           if r["port"] in PORT_HINTS]

async def fetch(ip: str, port: int):
```

```python
    proto = "https" if port in (443, 8443) else "http"
    url   = f"{proto}://{ip}:{port}/"
    try:
        async with httpx.AsyncClient(http2=True, verify=False, timeout=4) as cli:
            r  = await cli.head(url, follow_redirects=True)
            srv = r.headers.get("server", "").strip()
            title = ""
            if not srv:
                r2 = await cli.get(url, follow_redirects=True)
                if "captcha" in r2.text.lower():
                    return None
                m = re.search(r"<title.*?>(.*?)</title>", r2.text, re.S | re.I)
                title = m.group(1).strip()[:150] if m else ""
            banner = srv or title or f"{r.status_code}"
            return {"ip": ip, "port": port, "banner": banner}
    except Exception:
        return None

async def main():
    sem, rows = asyncio.Semaphore(400), []
    async def worker(ip, port):
        async with sem:
            res = await fetch(ip, port)
            if res:
                rows.append(res)

    await asyncio.gather(*(worker(ip, p) for ip, p in track(targets, description="HTTP")))
    if rows:
        with OUT_CSV.open("w", newline="") as fh:
            csv.DictWriter(fh, fieldnames=rows[0].keys()).writerows(rows)
        print(f"[+] HTTP banners → {OUT_CSV}")

asyncio.run(main())
```

Annotated design decisions

- **http2=True** – a lot of edge gateways require ALPN `h2`; we stay modern.
- Skip pages that visually include *captcha* to cut noise and avoid CAPTCHA solves.
- Title trimmed to 150 chars—enough for dashboards, without ballooning CSV.

3.4.4 Import Everything into SQLite (`import_enum.py`)

```
#!/usr/bin/env python3
"""
import_enum.py
--------------
Creates (or updates) `recon.db` with enumeration CSVs.
SQLite schema:
  ports(ip, port)  ← from 3.2
  tls(...)
  ssh(...)
  http(...)
User can run repeatedly; replaces tables atomically.
"""

import pandas as pd, sqlite3, pathlib

DB = sqlite3.connect("recon.db")

# Base ports (already imported earlier, keep as is)
# DB.execute("CREATE TABLE IF NOT EXISTS ports ...")

# Replace or insert protocol tables
for csv in ("tls.csv", "ssh.csv", "http.csv"):
    fp = pathlib.Path(csv)
```

```
    if fp.exists():
        df = pd.read_csv(fp)
        df.to_sql(fp.stem, DB, if_exists="replace", index=False)
        print(f"[+] {fp.stem} table upserted ({len(df)}) rows")

DB.close()
```

Everything is now queryable:

```
SELECT ip, port, banner
FROM http
WHERE banner LIKE '%WordPress%'
  AND ip IN (SELECT ip FROM tls WHERE ja3s='cfc62...');
```

3.4.5 Operating-Safely Summary

- **Semaphore caps** (250 TLS, 300 SSH, 400 HTTP) keep FD usage predictable.
- Every script exits cleanly on `Ctrl-C` *and* never leaves partially written CSV—open CSV handles are flushed after gather completes.
- Probe scripts respect `stop.txt`; check once at top of each worker loop if you extend them.

3.4.6 Self-study tasks

1. **SMB detector** – extend with `impacket.smb` to log `NativeOS` and `Dialect`.
2. **Mongo handshake** – send `\x3a\x00\x00\x00...isMaster` OP_MSG probe; parse JSON reply for version.
3. **ALPN anomaly** – SQL query: list TLS endpoints where ALPN is empty **and** port 443 (possible non-HTTP service).
4. **Weak RSA** – find SSH keys < 2048 bits; generate report for remediation.
5. **Parquet export** – write `enum.parquet` combining all tables for fast Pandas analysis.

3.5 - Analytics & Reporting

*"If you can't **explain** the data, you never really collected it."*

> **Starting point:** the unified database `recon.db` produced in 3.4
> **End product:** a one-click HTML/CSV report that highlights high-risk findings, plus a handful of reusable SQL/Pandas snippets the operators can run ad-hoc.

3.5.0 Goals for this stage

1. **Normalize** all enumeration tables into analyst-friendly views (no JOIN brain-twisters).
2. **Score** each service for risk using simple heuristics (weak key, outdated version, strange JA3).
3. **Export** a single HTML dashboard (*and* raw CSVs) that managers and pentesters can both read.
4. **Keep it portable**—pure Python; no Tableau, no Grafana installs.

3.5.1 Library stack

Purpose	Library	Why
SQL ↔ DataFrame bridge	pandas + sqlite3	Already used in 3-series
HTML templating	Jinja2	Zero JS, renders to flat file
Risk scoring helper	plain re / packaging	Regex version grab, semantic compare
(Optional) sparkline	pandas-styler	Adds colour bars without JS

```
pip install pandas jinja2 packaging
```

3.5.2 Database recap

`recon.db` should now contain:

- `ports` (ip, port) – from 3.2
- `nmap` (host, portid, service, product…) – from 3.3
- `tls` (ip, port, ja3s, alpn, cn, cert_not_after)
- `ssh` (ip, port, banner, algo, bits)
- `http` (ip, port, banner)

The queries below assume this schema.

3.5.3 Risk-scoring logic (explain first)

Rule ID	Condition	Score	Rationale
R01	TLS certificate expires in < 30 days	2 pts	Impending outage or sloppy cert mgmt
R02	SSH host-key < 2048 bits or algorithm dss	3 pts	Weak cryptography
R03	HTTP banner contains strings "PHP/5.", "Apache/2.2"	2 pts	End-of-life software
R04	TLS ALPN list **empty** on port 443	1 pt	Non-HTTP service on HTTPS port (misconfiguration)
R05	JA3S hash in known WAF/CDN list (could hide real origin)	1 pt	Extra pivot value
Total	Sum (0–9 points) → Low (0-2), Med (3-5), High (6+)		

3.5.4 `report.py` – fully commented

```python
#!/usr/bin/env python3
"""
report.py
---------
Reads recon.db, applies risk rules, writes:
  • risk.csv   (flat table)
  • summary.html (clickable dashboard)
"""

from __future__ import annotations
import sqlite3, pandas as pd, re
from packaging.version import parse as vparse
from jinja2 import Environment, FileSystemLoader
from datetime import datetime, timedelta
from pathlib import Path

DB = sqlite3.connect("recon.db")

##############################################################################
# 1. Load all tables into DataFrames
##############################################################################
ports = pd.read_sql("SELECT * FROM ports", DB)
nmap  = pd.read_sql("SELECT * FROM nmap",  DB)
tls   = pd.read_sql("SELECT * FROM tls",  DB)
ssh   = pd.read_sql("SELECT * FROM ssh",  DB)
http  = pd.read_sql("SELECT * FROM http", DB)

##############################################################################
# 2. Merge into one wide table keyed by ip+port
##############################################################################
def merge_two(left, right):
    return pd.merge(left, right, how="left",
                    left_on=["ip", "port"],
```

```
                    right_on=["ip", "port"])

merged = ports.copy()
for df in (nmap, tls, ssh, http):
    merged = merge_two(merged, df)

merged.fillna("", inplace=True)   # easier string ops

################################################################################
# 3. Risk-scoring functions
################################################################################
def r01_cert_exp(row):
    if row["cert_not_after"]:
        exp = datetime.strptime(row["cert_not_after"], "%Y-%m-%dT%H:%M:%SZ")
        return exp < datetime.utcnow() + timedelta(days=30)
    return False

def r02_ssh_weak(row):
    return row.get("bits", 9999) and int(row["bits"] or 4096) < 2048 or row["algo"].startswith("ssh-dss")

def r03_eol_http(row):
    banner = row["banner"].lower()
    return any(x in banner for x in ("php/5.", "apache/2.2"))

def r04_alpn_empty(row):
    return row["port"] == 443 and not row["alpn"]

WAF_HASHES = {"d32b5b65d7c53c9b0ffd175bb1a4c148", "e2ab4b05095c46c2e6b6e65caf35e26a"}  # sample

def r05_waf(row):
    return row["ja3s"] in WAF_HASHES

# Attach columns
merged["score"] = 0
```

```
for rule, weight in [(r01_cert_exp, 2),
                     (r02_ssh_weak, 3),
                     (r03_eol_http, 2),
                     (r04_alpn_empty, 1),
                     (r05_waf, 1)]:
    merged["score"] += merged.apply(rule, axis=1) * weight

merged["severity"] = pd.cut(merged["score"],
                            bins=[-1,2,5,99],
                            labels=["Low","Medium","High"])

##############################################################################
# 4. Export CSV
##############################################################################
out_csv = Path("risk.csv")
merged.to_csv(out_csv, index=False)
print(f"[+] Flat risk table → {out_csv}")

##############################################################################
# 5. HTML Summary via Jinja2 template
##############################################################################
# Simple template lives inline for brevity
template_str = """
<!DOCTYPE html>
<html>
<head>
  <title>Recon Risk Report</title>
  <style>
    body{font-family:sans-serif}
    table{border-collapse:collapse;width:100%}
    th,td{border:1px solid #ccc;padding:6px;font-size:13px}
    .High{background:#ffb3b3}.Medium{background:#ffe0b3}.Low{background:#e0ffe0}
  </style>
</head>
```

```
<body>
<h1>Recon Risk Report — {{ date }}</h1>
<p>Total services: {{ total }} — High: {{ high }} / Medium: {{ medium }} / Low: {{ low }}</p>
<table>
  <tr><th>IP</th><th>Port</th><th>Service</th><th>Banner / CN</th><th>Score</th><th>Severity</th></tr>
  {% for r in rows %}
  <tr class="{{ r.severity }}">
    <td>{{ r.ip }}</td><td>{{ r.port }}</td>
    <td>{{ r.service|default('') }}</td>
    <td>{{ r.banner or r.cn }}</td>
    <td>{{ r.score }}</td><td>{{ r.severity }}</td>
  </tr>
  {% endfor %}
</table>
</body></html>
"""
env = Environment(loader=FileSystemLoader("."))  # dummy
tmpl = env.from_string(template_str)

html_out = Path("summary.html")
html_out.write_text(tmpl.render(
    date=datetime.utcnow().strftime("%Y-%m-%d %H:%M UTC"),
    total=len(merged),
    high=(merged.severity=="High").sum(),
    medium=(merged.severity=="Medium").sum(),
    low=(merged.severity=="Low").sum(),
    rows=merged.sort_values("score", ascending=False).to_dict("records")[:500]  # top 500 rows
))
print(f"[+] HTML dashboard → {html_out.resolve()}")
```

How the comments explain decisions

- **Rule functions** are Python callables returning bool; easy to extend.

- We label severities via `pd.cut` instead of nested `if` chains—cleaner.
- HTML template is inlined for copy-paste convenience; external file works too.
- Only **top 500** rows render to HTML to keep the file light (< 1 MB).

3.5.5 Example analyst queries (ready-to-copy SQL)

```sql
-- Weak SSH keys by algorithm
SELECT ip, port, algo, bits
FROM ssh
WHERE bits < 2048 OR algo LIKE 'ssh-dss%';

-- TLS certs about to expire
SELECT ip, port, cert_not_after
FROM tls
WHERE date(cert_not_after) < date('now', '+30 day');

-- Top 10 HTTP banners
SELECT banner, COUNT(*) c
FROM http
GROUP BY banner
ORDER BY c DESC
LIMIT 10;
```

3.5.6 Optional: interactive notebook

If you prefer a Jupyter workflow, launch:

```
jupyter lab --notebook-dir reports/
```

and import `risk.csv`:

```
import pandas as pd
df = pd.read_csv("risk.csv")
df[df["severity"]=="High"].head()
```

3.5.7 Exercises

1. **Heat-map** – Join `risk.csv` with MaxMind's GeoLite country code and produce a choropleth via `folium`.
2. **Slack alert** – Post any new "High" issues to a Slack channel via webhook.
3. **Delta diff** – Compare today's `risk.csv` with yesterday's and list newly exposed services. (Hint: use Pandas `merge` with `indicator=True`).
4. **PDF exporter** – Wrap the HTML in `pdfkit` (wkhtmltopdf) to generate a shareable PDF.
5. **Custom rule** – Add R06: flag any `nmap.service` that equals `"ftp"` **and** banner contains `"vsftpd 2.3.4"` (known backdoor).

3.5.8 Done checklist

- `risk.csv` generated, non-empty.
- `summary.html` opens locally with colour-coded rows.
- Sample SQL queries return expected results.
- Rule weights easy to tweak; re-running `report.py` refreshes outputs.

1. What Python module is commonly used for raw TCP socket scanning?

- ◯ A) socket
- ◯ B) urllib
- ◯ C) os
- ◯ D) platform

A resposta correta é:

✅ **A) socket**

Explicação

O módulo padrão do Python usado para **comunicação de rede em baixo nível** é o **socket**.

Ele permite criar conexões TCP/UDP diretamente e é amplamente usado para:

- **port scanning**
- **banner grabbing**
- **network probing**
- **custom scanners**

Exemplo simples de **TCP port scan** em Python:

```
import socket

target = "192.168.1.1"
port = 80

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
s.settimeout(1)

result = s.connect_ex((target, port))

if result == 0:
    print("Port open")
else:
    print("Port closed")

s.close()
```

Por que as outras opções estão erradas?

B) urllib

❌ Usado para **requisições HTTP**, não para sockets TCP brutos.

C) os

❌ Usado para interação com o sistema operacional (files, env vars, processos).

D) platform

❌ Usado para obter informações do sistema (OS, arquitetura, etc.).

Regra rápida

Módulo	Uso
socket	rede TCP/UDP
urllib	HTTP requests
os	sistema operacional
platform	info do sistema

✅ **Resposta final: A**

2. Which flag combination is typically used in a TCP SYN scan?

- ◯ A) FIN
- ◯ B) SYN
- ◯ C) ACK
- ◯ D) RST

A resposta correta é:

✅ **B) SYN**

Explicação

Um **TCP SYN scan** (também chamado de **half-open scan**) funciona enviando apenas o **flag SYN** para iniciar o handshake TCP.

O comportamento esperado é:

Resposta do alvo	Significado
SYN-ACK	Porta **aberta**
RST	Porta **fechada**
Sem resposta / ICMP	Porta **filtrada**

Fluxo simplificado:

```
Scanner → SYN → Target
Target → SYN-ACK → Scanner
Scanner → RST → Target
```

O scanner envia **RST** para **não completar o handshake**, por isso o scan é chamado de **half-open**.

Por que as outras estão erradas?

A) FIN

❌ Usado em **FIN scan**, não SYN scan.

C) ACK

❌ Usado em **ACK scan** para mapear regras de firewall.

D) RST

❌ Usado para encerrar conexões.

Regra rápida para lembrar

Scan	Flag usada
SYN scan	**SYN**
FIN scan	FIN
ACK scan	ACK
Xmas scan	FIN + PSH + URG

Ferramentas como **Nmap** usam esse método porque ele é **mais rápido e menos detectável** que o connect scan completo.

✅ **Resposta final: B) SYN**

3. Which Python library allows asynchronous port scanning?

- A) requests
- B) multiprocessing
- C) asyncio
- D) yaml

A resposta correta é:

✅ **C) asyncio**

Explicação

O **asyncio** é a biblioteca padrão do Python usada para **programação assíncrona baseada em event loop**, ideal para tarefas de rede que precisam lidar com **muitas conexões simultâneas**, como:

- **port scanning**
- crawlers
- scanners de rede
- coleta OSINT
- scraping em larga escala

Em vez de criar milhares de threads, o asyncio permite executar **muitas operações de I/O simultaneamente** de forma eficiente.

Exemplo simples de port scan assíncrono

```
import asyncio
```

```
async def scan_port(host, port):
    try:
        reader, writer = await asyncio.open_connection(host, port)
        print(f"Port {port} open")
        writer.close()
        await writer.wait_closed()
    except:
        pass

async def main():
    tasks = [scan_port("192.168.1.1", p) for p in range(1,1024)]
    await asyncio.gather(*tasks)

asyncio.run(main())
```

Esse código consegue testar **centenas de portas rapidamente** usando concorrência assíncrona.

Por que as outras estão erradas?

A) requests

❌ Biblioteca HTTP síncrona.

B) multiprocessing

❌ Usa processos paralelos, não programação assíncrona baseada em event loop.

D) yaml

❌ Usado para ler arquivos de configuração YAML.

Regra rápida

Biblioteca	Uso
asyncio	I/O assíncrono
multiprocessing	paralelismo com processos
requests	HTTP simples
yaml	parsing de configuração

✅ **Resposta final: C) asyncio**

4. What is the default port for HTTPS?

- ◯ A) 443
- ◯ B) 80
- ◯ C) 22
- ◯ D) 21

A resposta correta é:

✅ **A) 443**

Explicação

O **HTTPS (Hypertext Transfer Protocol Secure)** utiliza por padrão a **porta TCP 443**.

HTTPS é basicamente **HTTP sobre TLS/SSL**, garantindo:

- **criptografia**
- **integridade**
- **autenticação do servidor**

Fluxo simplificado:

```
Client → TCP 443 → Server
        TLS Handshake
        HTTP Request (encrypted)
```

Portas comuns de serviços

Serviço	Porta
HTTP	80
HTTPS	**443**
SSH	22
FTP	21

Por que as outras estão erradas?

B) 80

❌ Porta padrão do **HTTP**.

C) 22

❌ Porta padrão do **SSH**.

D) 21

❌ Porta padrão do **FTP**.

✅ **Resposta final: A) 443**

5. What does a closed port typically respond with?

- ◯ A) SYN-ACK
- ◯ B) RST
- ◯ C) FIN
- ◯ D) Nothing

A resposta correta é:

✅ **B) RST**

Explicação

Quando um scanner envia um **SYN** para uma porta **fechada**, o host responde com **RST (Reset)** para indicar que **não existe serviço escutando naquela porta**.

Fluxo típico em um **SYN scan**:

Porta aberta

```
Scanner → SYN → Target
Target → SYN-ACK → Scanner
Scanner → RST → Target
```

Porta fechada

```
Scanner → SYN → Target
Target → RST → Scanner
```

Significado das respostas

Resposta	Significado
SYN-ACK	Porta **aberta**
RST	Porta **fechada**
Sem resposta / ICMP unreachable	Porta **filtrada** (firewall)

Por que as outras estão erradas?

A) SYN-ACK

❌ Indica **porta aberta**.

C) FIN

❌ Usado para **encerrar conexões TCP**.

D) Nothing

❌ Geralmente indica **porta filtrada**, não fechada.

✅ **Resposta final: B) RST**

6. What tool inspired the creation of Python-based service enumerators?

- A) Burp Suite
- B) Nikto
- C) Nessus
- D) Nmap

A resposta correta é:

✅ **D) Nmap**

Explicação

O **Nmap** é a ferramenta clássica de **port scanning e service enumeration**, e serviu de inspiração para muitos scanners e enumeradores escritos em Python.

O Nmap implementa técnicas como:

- **Port scanning**
- **Service detection (**-sV**)**
- **OS fingerprinting**
- **Scriptable enumeration (NSE)**

Exemplo típico:

```
nmap -sV 192.168.1.10
```

Saída:

```
22/tcp  open  ssh     OpenSSH 8.2
80/tcp  open  http    Apache httpd 2.4.41
443/tcp open  https   nginx 1.18
```

Ferramentas Python frequentemente **replicam partes desse comportamento**, por exemplo:

- banner grabbing
- detecção de versão
- enumeração de serviços
- scans assíncronos

Por que as outras estão erradas?

A) Burp Suite

❌ Ferramenta para **web application testing**.

B) Nikto

❌ Scanner de **vulnerabilidades web**.

C) Nessus

❌ Scanner de **vulnerabilidades corporativo**, não focado em service enumeration.

✅ **Resposta final: D) Nmap**

7. In banner grabbing, what method is often used?

- ◯ A) Send partial HTTP requests
- ◯ B) Inject shellcode
- ◯ C) Spoof DNS
- ◯ D) Use ICMP probes

A resposta correta é:

✅ **A) Send partial HTTP requests**

Explicação

Banner grabbing consiste em enviar uma **requisição mínima ou parcial** a um serviço para fazer o servidor revelar informações sobre si mesmo.

No caso de serviços HTTP, um método comum é enviar uma requisição simples como:

```
HEAD / HTTP/1.0
```

ou

```
GET / HTTP/1.0
```

O servidor normalmente responde com **headers que revelam o software e versão**.

Exemplo de resposta:

```
HTTP/1.1 200 OK
Server: Apache/2.4.54 (Ubuntu)
X-Powered-By: PHP/8.1
```

Isso permite identificar:

- servidor web (Apache, Nginx, IIS)
- versão do software
- frameworks usados

Por que as outras estão erradas?

B) Inject shellcode

❌ Isso é **exploração**, não enumeração.

C) Spoof DNS

❌ Técnica de ataque de rede, não banner grabbing.

D) Use ICMP probes

❌ ICMP é usado para **host discovery (ping)**.

✅ **Resposta final: A) Send partial HTTP requests**

8. Which of the following is NOT a valid port number?

- A) 1
- B) 22
- C) 70000
- D) 65535

A resposta correta é:

✅ **C) 70000**

Explicação

Os **números de portas TCP/UDP** são definidos em um campo de **16 bits**, então o intervalo válido é:

0 ; \text{até} ; 65535

Portanto:

Porta	Válida?
1	✅ válida
22	✅ válida (SSH)
65535	✅ válida (maior porta possível)
70000	❌ **inválida**

Classificação das portas

Intervalo	Tipo
0–1023	Well-known ports
1024–49151	Registered ports
49152–65535	Dynamic / Ephemeral

Exemplos conhecidos:

Serviço	Porta
HTTP	80
HTTPS	443
SSH	22
DNS	53

✅ **Resposta final: C) 70000**

9. What response indicates a port is filtered?

- ◯ A) RST
- ◯ B) FIN
- ◯ C) No response
- ◯ D) ICMP Unreachable

A resposta correta é:

✅ **C) No response**

Explicação

Durante um **port scan**, uma porta **filtrada** geralmente indica que um **firewall ou IDS/IPS está bloqueando o tráfego**.

Nesse caso, o comportamento típico é:

```
Scanner → SYN → Target
Firewall → (drop packet)
Scanner → timeout (no response)
```

Ou seja, **nenhuma resposta é recebida**.

Interpretação das respostas em scans

Resposta	Significado
SYN-ACK	Porta **aberta**
RST	Porta **fechada**
No response	Porta **filtrada** (firewall)

Sobre ICMP Unreachable

Alguns firewalls podem retornar **ICMP unreachable**, mas na prática muitos simplesmente **descartam o pacote**, resultando em **nenhuma resposta**, que é o comportamento mais típico considerado em scans.

Por que as outras estão erradas?

A) RST

❌ Indica **porta fechada**.

B) FIN

❌ Flag usada para **encerrar conexão**, não resposta de scan.

D) ICMP Unreachable

⚠️ Pode ocorrer em alguns casos, mas o comportamento **mais típico** de filtragem é **não responder**.

✅ **Resposta final: C) No response**

10. What is the main risk when doing active port scanning?

- ◯ A) Wasting bandwidth
- ◯ B) Getting detected
- ◯ C) Slow speed
- ◯ D) High CPU usage

A resposta correta é:

✅ **B) Getting detected**

Explicação

Active port scanning envia pacotes diretamente para o alvo (SYN, TCP connect, UDP probes, etc.).

Isso gera **tráfego detectável na rede**, que pode ser registrado por:

- **Firewalls**
- **IDS/IPS**
- **SIEM**
- **EDR/XDR**

Exemplo de detecção comum:

```
Alert: Port scan detected
Source: 10.10.10.5
Destination: 192.168.1.20
Ports scanned: 1–1000
```

Ferramentas de segurança como:

- **Snort**
- **Suricata**
- **Zeek**
- **SIEMs corporativos**

possuem **regras específicas para detectar port scans**.

Por que as outras estão erradas?

A) Wasting bandwidth

❌ O tráfego de scan normalmente é pequeno.

C) Slow speed

❌ Scanners modernos podem ser muito rápidos.

D) High CPU usage

❌ Não é o principal risco.

Conceito importante

Tipo de Recon	Risco
Passive recon	baixo risco de detecção
Active recon	**alto risco de detecção**

✅ **Resposta final: B) Getting detected**